

1 Хеширование строк

1.1 Домашнее задание

Определение. Семейство хеш-функций $\mathcal{H} = \{h : X \rightarrow Y\}$ называется

- универсальным, если:

$$\forall x_1, x_2 \in X, x_1 \neq x_2 : \Pr_{h \in \mathcal{H}} [h(x_1) = h(x_2)] \leq \frac{1}{|Y|}$$

- k -независимым, если для любых различных $x_1, x_2, \dots, x_k \in X$, для любых, возможно, совпадающих $y_1, y_2, \dots, y_k \in Y$ выполняется:

$$\Pr_{h \in \mathcal{H}} \left[\bigwedge_{i=1}^k h(x_i) = y_i \right] = \frac{1}{|Y|^k}$$

1. (а) Докажите, что любое 2-независимое семейство хэш-функций является универсальным.
(б) Докажите, что любое $k+1$ -независимое семейство хэш-функций является k -независимым.
2. Придумайте строку длины n над алфавитом $\{0, 1\}$, в которой $\Omega(n^2)$ различных подстрок.
3. Даны две строки суммарной длины n . Найдите их самую длинную общую подстроку. $\mathcal{O}(n \log n)$.
4. Даны строка s длины n и число k . Найдите в s за время $\mathcal{O}(n)$ самую длинную подстроку, представимую как степень некоторой строки x , для которой $|x| = k$ (степенью строки называют её конкатенацию с собой несколько раз).
5. Найти подстроку в тексте. При сравнении строк, если несовпадений символов было не более k , строки считаются равными. $\mathcal{O}(nk \log n)$.
6. Дана строка длины n . Запросы:
 - `set(i, c)` – установить символ на i -й позиции в s .
 - `test(i, j, l)` – проверить на равенство подстроки длины l начинающиеся в i -й и j -й позициях.

$\mathcal{O}(\log n)$ на запрос, online.

Дополнительные задачи

7. Напишите на `Haskell` определение бесконечного списка `Int`-ов (из $\{0, 1\}$), представляющего собой последовательность Туэ-Морса (последовательность из задачи 7 практики) S_∞ , такое, что:
 - определение содержит не более 128 символов
 - можно использовать стандартные функции из модулей `Prelude` и `Data.List`
 - вычисление в нормальную форму `genericTake n` от списка занимает время $\mathcal{O}(n)$.
8. Пусть дана w -разрядная RAM машина. Для простоты можно считать w степенью двойки. Придумайте для такой машины детерминированный подсчет за $\mathcal{O}(w)$ операций, который позволит по w -битному числу вида 2^k восстанавливать k за $\mathcal{O}(1)$ операций.

1.2 Решения

1. (а) Пусть $\mathcal{H}$ — 2-независимое семейство хеш-функций.

Для любых различных $x_1, x_2 \in X$ вероятность коллизии:

$$\Pr_{h \in \mathcal{H}} [h(x_1) = h(x_2)] = \sum_{y \in Y} \Pr_{h \in \mathcal{H}} [h(x_1) = y \wedge h(x_2) = y].$$

Из 2-независимости для любого $y \in Y$:

$$\Pr_{h \in \mathcal{H}} [h(x_1) = y \wedge h(x_2) = y] = \frac{1}{|Y|^2}$$

Суммируя по всем y , получаем:

$$\sum_{y \in Y} \frac{1}{|Y|^2} = \frac{|Y|}{|Y|^2} = \frac{1}{|Y|}.$$

Таким образом, мы доказали универсальность $\mathcal{H}$.

- (b) Пусть $\mathcal{H}$ — $(k+1)$ -независимое семейство.

Возьмём произвольные различные $x_1, \dots, x_k \in X$ и $y_1, \dots, y_k \in Y$. Добавим произвольный $x_{k+1} \neq x_1, \dots, x_k$.

Тогда:

$$\Pr_{h \in \mathcal{H}} \left[\bigwedge_{i=1}^k h(x_i) = y_i \right] = \sum_{y_{k+1} \in Y} \Pr_{h \in \mathcal{H}} \left[\bigwedge_{i=1}^{k+1} h(x_i) = y_i \right].$$

Из $(k+1)$ -независимости каждое слагаемое равно $\frac{1}{|Y|^{k+1}}$.

Суммируя по $|Y|$ значениям y_{k+1} :

$$\sum_{y_{k+1} \in Y} \frac{1}{|Y|^{k+1}} = \frac{|Y|}{|Y|^{k+1}} = \frac{1}{|Y|^k}.$$

Следовательно, $\mathcal{H}$ является k -независимым.

2. Рассмотрим строку вида $0^{\lfloor n/2 \rfloor} 1^{\lceil n/2 \rceil}$

Таким образом, рассмотрим количество различных подстрок в различных частях строки:

- (а) 0^i равно i .
- (b) 1^i равно i .
- (с) $0^i 1^j$ равно $i \cdot j$.

Получаем $i + j + i \cdot j \geq \frac{n^2}{4}$, что является $\Omega(n^2)$.

3. Будем использовать бинарный поиск по длине общей подстроки. Пусть l — текущая проверяемая длина подстроки.

Для проверки существования общей подстроки длины l за $O(n)$ предварительно построим префиксные хеши для обеих строк. Это позволит находить хеш любой подстроки длины l за $O(1)$.

Проверка для фиксированного l осуществляется следующим образом:

- (а) Для первой строки вычисляем хеши всех подстрок длины l и сохраняем их в множество.
- (b) Для второй строки проверяем, существует ли хотя бы один из её хешей подстрок длины l в этом множестве.

Если хотя бы одна общая подстрока найдена, увеличиваем левую границу бинарного поиска, иначе уменьшаем правую. Чтобы минимизировать вероятность коллизий, можно использовать двойное хеширование.

Сложность бинарного поиска $O(\log n)$, а проверка каждой длины $O(n)$, итого $O(n \log n)$.

4. Предварительно построим префиксные хеши для строки s . Будем использовать динамическое программирование и sliding window для нахождения ответа.

Введём массив dp размера $n - k + 1$, где $dp[i]$ — количество последовательных одинаковых подстрок длины k , начинающихся/продолжающихся в i . Изначально $dp[i] = 0$.

Для каждого $i \in [0, n - k)$ вычислим хеш подстроки $s[i, i + k)$. Если $i \geq k$, дополнительно сравним его с хешем предыдущей подстроки $s[i - k, i)$:

- Если хеши совпадают, значит, подстрока продолжает последовательность, и тогда:

$$dp[i] = dp[i - k] + 1.$$

- Иначе начинается новая последовательность:

$$dp[i] = 1.$$

Ответом будет максимальное значение в массиве dp , а саму подстроку можно восстановить по индексу.